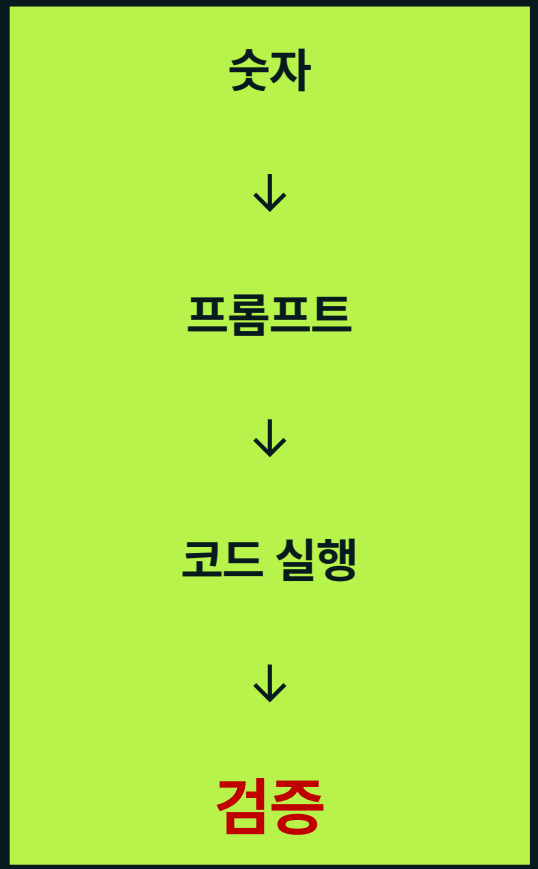
스마트 팩토리 입문
Claude Code 워크북

같은 다섯 예제를 프롬프트로 풀고, 손계산 값으로 검증한다



스마트 팩토리 1일 과정 · 워크북 II

손으로 푼 다섯 예제를 프롬프트로 다시 풉니다

예제마다 프롬프트 → 실행 결과 → 후속 프롬프트(확인 문제) 순서로 진행한다

01	OEE	**oee.py** 세 비율 · assert 검산 · 워터폴 그래프
02	교체 기준 확률	**decide(cp, cf, p)** 기대 비용 · p* · 비용선 그래프
03	RMS · 첨도	**vib.py** 단계별 RMS · 신호 시뮬레이션
04	Weibull 생존 확률	**weibull.py** R(500) · R(1000) · 중앙 수명
05	NPV · 민감도	**npv.py** 할인 표 · 시나리오 · 손익분기

Claude Code는 계산을 코드로 바꿔 직접 실행합니다

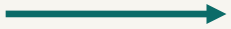
Claude Code — 터미널에서 일하는 AI 에이전트. 폴더 안에서 파일을 만들고 코드를 돌린다

손계산

원리를 몸에 익힌다

- 한 문제씩 손으로
- 숫자를 바꾸면 다시
- 그래프는 그리기 어려움

왜 그런지 이해



CLAUDE CODE

같은 계산을 반복한다

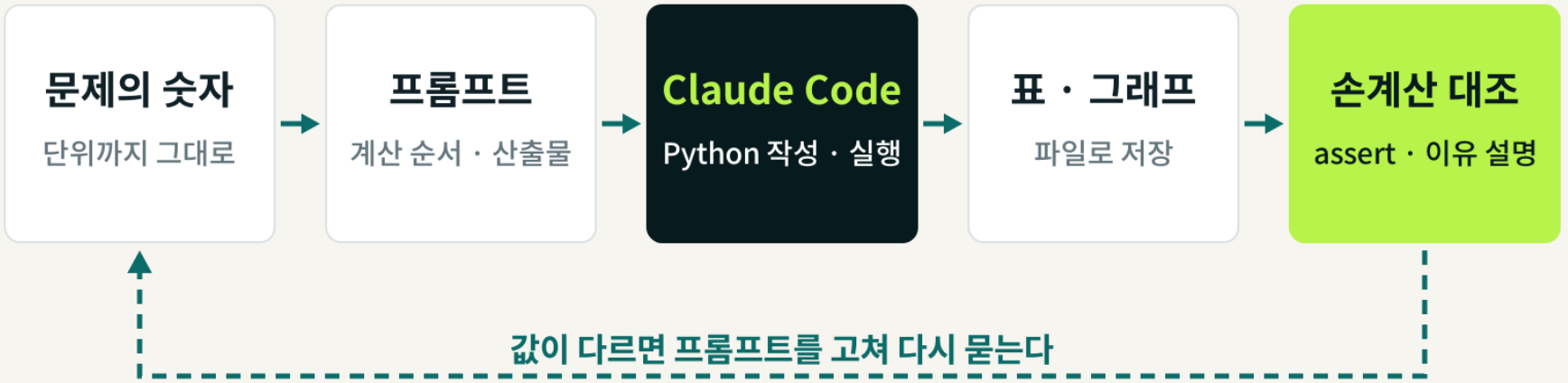
- Python을 작성·실행
- 숫자만 바꿔 재실행
- 표·그래프를 파일로

빠르게, 많이, 기록으로

손계산으로 답을 알고 있어야 Claude Code의 답을 검증할 수 있습니다

한 문제는 다섯 단계의 고리로 풉니다

숫자를 넣고, 순서를 정하고, 실행하고, 파일로 받고, 손계산과 맞춰 본다



※ 값이 다르면 Claude Code를 의심하기 전에 프롬프트의 숫자·단위부터 다시 본다

좋은 프롬프트는 여섯 칸을 채웁니다

이후 모든 예제의 프롬프트가 이 템플릿을 따른다

재사용 프롬프트 템플릿

```
> [역할]   너는 제조 설비 데이터 분석가다.
  [데이터] 숫자·파일 경로를 그대로 붙인다 (단위 포함)
  [계산]   1) 2) 3) 순서로 — 한 번에 한 가지씩
  [산출물] 표는 소수 자리, 그림은 파일 이름까지 지정
  [검증]   손계산 값과 assert로 대조, 다르면 이유 설명
  [보관]   코드를 .py로 남겨 다음 문제에 재사용
```

※ 한 번에 여러 일을 시키지 말고 1) 2) 3)으로 나눠야 결과를 단계별로 확인할 수 있다

예제 1. 다섯 숫자를 주고 OEE와 검산을 함께 시킵니다

계산 → 검산(assert) → 그래프 → 코드 보관 순서로 요청한다

Claude Code · 예제 1 OEE

```
> 사출성형기 하루 데이터로 OEE를 계산해줘.
  계획 480분 · 정지 60분 · 이론 사이클 1분/개
  생산 380개 · 양품 361개
1) 가동률·성능률·양품률·OEE를 % 소수 첫째 자리로 출력
2) 양품 ÷ 계획 시간으로 검산하고 assert로 확인
3) 손실 워터폴 막대그래프를 oee.png로 저장
계산 코드는 oee.py로 남겨줘.
```

※ assert — 계산값이 기대값과 다르면 프로그램을 멈추게 하는 검사 문장

실행 결과는 손계산과 같은 75.2%입니다

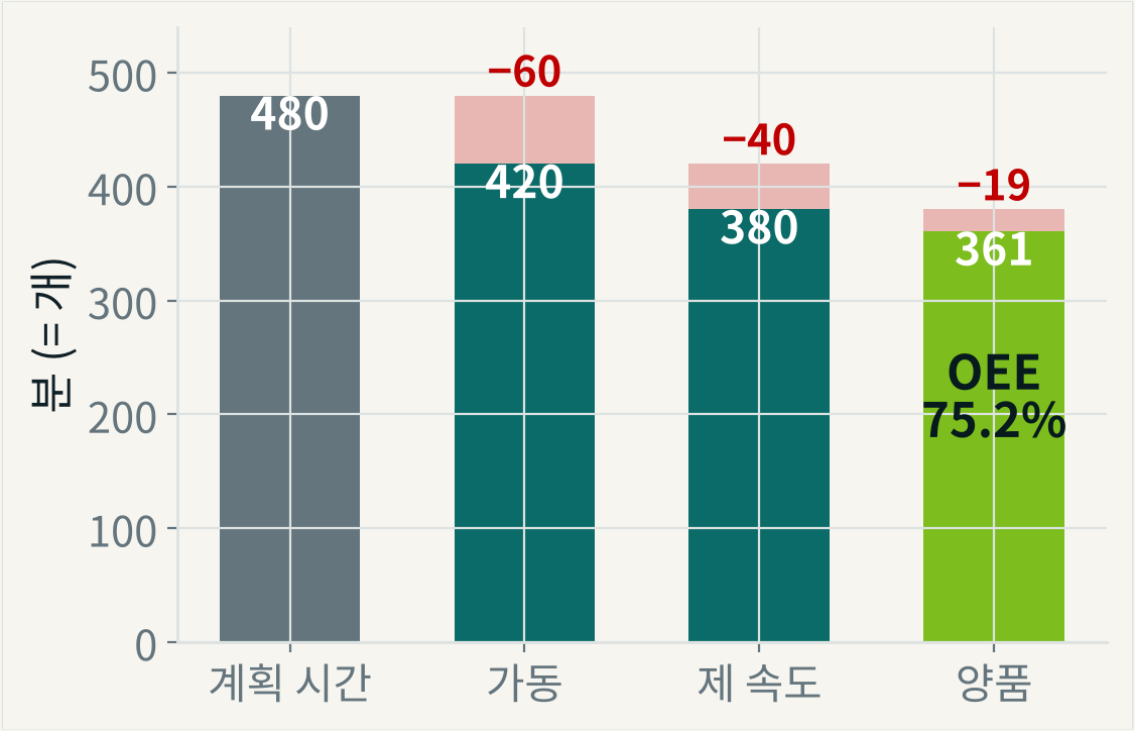
oee.py 실행 출력과 저장된 oee.png

출력

OEE 75.2%

- 가동률 87.5%
- 성능률 90.5%
- 양품률 95.0%

assert 통과



확인 문제 1. 함수를 재사용하면 비교가 한 줄로 끝납니다

A: 95% · 80% · 99%, B: 세 비율 모두 90% — 어느 쪽이 높고 어디부터 고칠까

실행 결과 · 해설

A 75.2% > B 72.9%

- A 최저: 성능률 80%
- B 최저: 셋 다 90%
- A는 성능률부터

평균이 같아도 곱은 다르다

후속 프롬프트 · 문제 1

```
> oee.py를 재사용해 두 설비를 비교해줘.
  A: 가동 95% · 성능 80% · 양품 99%
  B: 세 비율 모두 90%
OEE가 높은 설비와, 설비마다
가장 낮은 비율을 표로 정리하고
어디부터 개선할지 한 줄로 써줘.
```

예제 2. 판단 규칙을 함수로 만들어 두게 합니다

계획 교체 200만 원 · 고장 2,000만 원 · 예측 고장 확률 15%

Claude Code · 예제 2 교체 기준

```
> 예지보전 교체 판단을 계산해줘.
  계획 교체 비용 200만 원 · 고장 비용 2,000만 원
  AI가 예측한 1주 안 고장 확률 15%
  1) 기다릴 때 기대 비용과 교체 비용을 비교해 결론 출력
  2) 교체 기준 확률 p* = 교체 비용 ÷ 고장 비용
  3) 확률 0~30%의 두 비용선 그래프를 cost.png로
  decide(cp, cf, p) 함수로 만들어 재사용하게 해줘.
```

※ 함수로 남기면 비용이나 확률이 바뀔 때 숫자만 넣어 다시 판단할 수 있다

두 비용선이 10%에서 만나고, 15%는 교체 쪽입니다

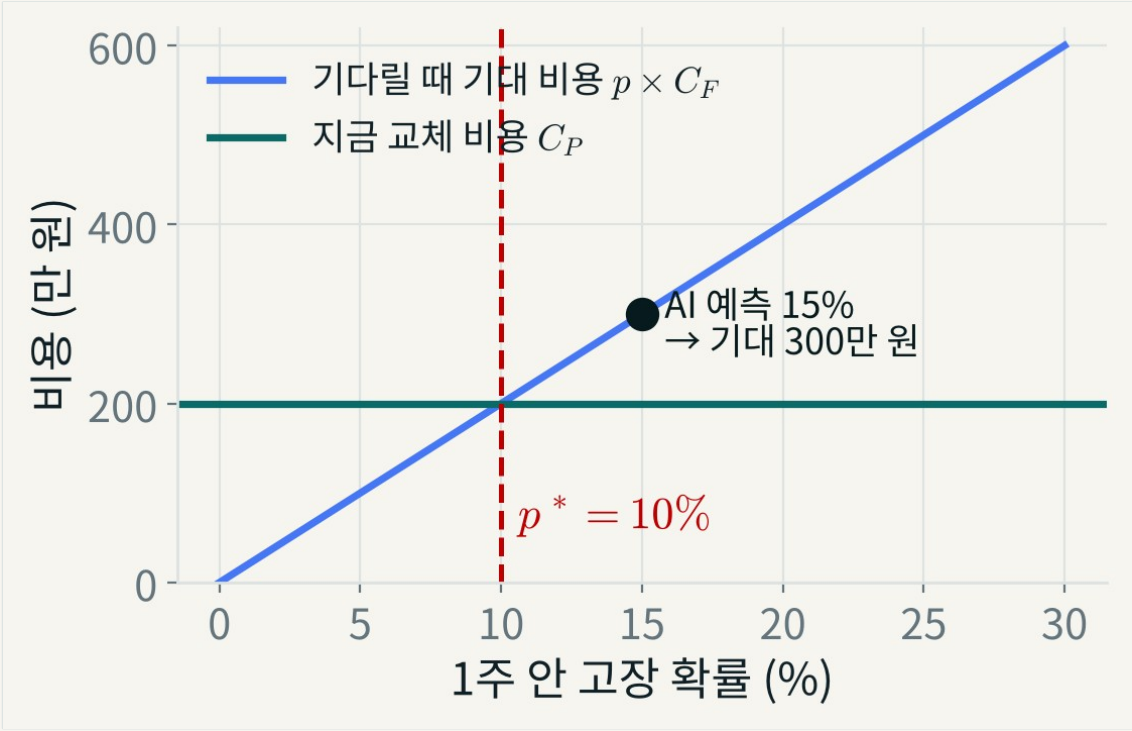
decide(200, 2000, 0.15) 실행 출력과 저장된 cost.png

출력

p* = 10%

- 기대 비용 300만
- 교체 비용 200만
- 결론: 지금 교체

손계산과 일치



확인 문제 2. 같은 함수에 숫자만 바꿔 넣습니다

교체 500만 원 · 고장 2,500만 원 · 예측 15% — 이번에는 기다리는가

실행 결과 · 해설

아직 기다린다

- p* = 20%
- 15% < 20%
- 기대 375만 < 500만

모니터링 강화

후속 프롬프트 · 문제 2

```
> decide()로 다시 판단해줘.
    교체 비용 500만 원
    고장 비용 2,500만 원
    예측 고장 확률 15%
  p*, 두 기대 비용, 결론을
  한 표로 출력해줘.
```

확인 문제 3. 경고선은 오경보 수와 함께 정합니다

60일 첨도 z값(예시 데이터)에서 경고선을 바꿔 가며 첫 경고일과 오경보를 센다

실행 결과 · 첫 경고일 · 오경보

- 1σ: 1일 · 3회
- 1.5σ: 10일 · 1회
- 2σ: 33일 · 0회
- 3σ: 36일 · 0회

2σ가 적당 — 3σ보다 3일 앞서 경고

후속 프롬프트 · 문제 3

```
> 60일 첨도 z값으로 경고선을
    1σ · 1.5σ · 2σ · 3σ로 바꿔 가며
    ① 첫 경고일
    ② 정상 구간(0~28일) 오경보 수
  를 표로 만들어줘.
  어느 경고선이 적당한지 근거도.
```

예제 3. 계산 단계를 그대로 출력하게 해서 따라갑니다

손계산 예제와 실제와 비슷한 신호 시뮬레이션을 한 번에 요청한다

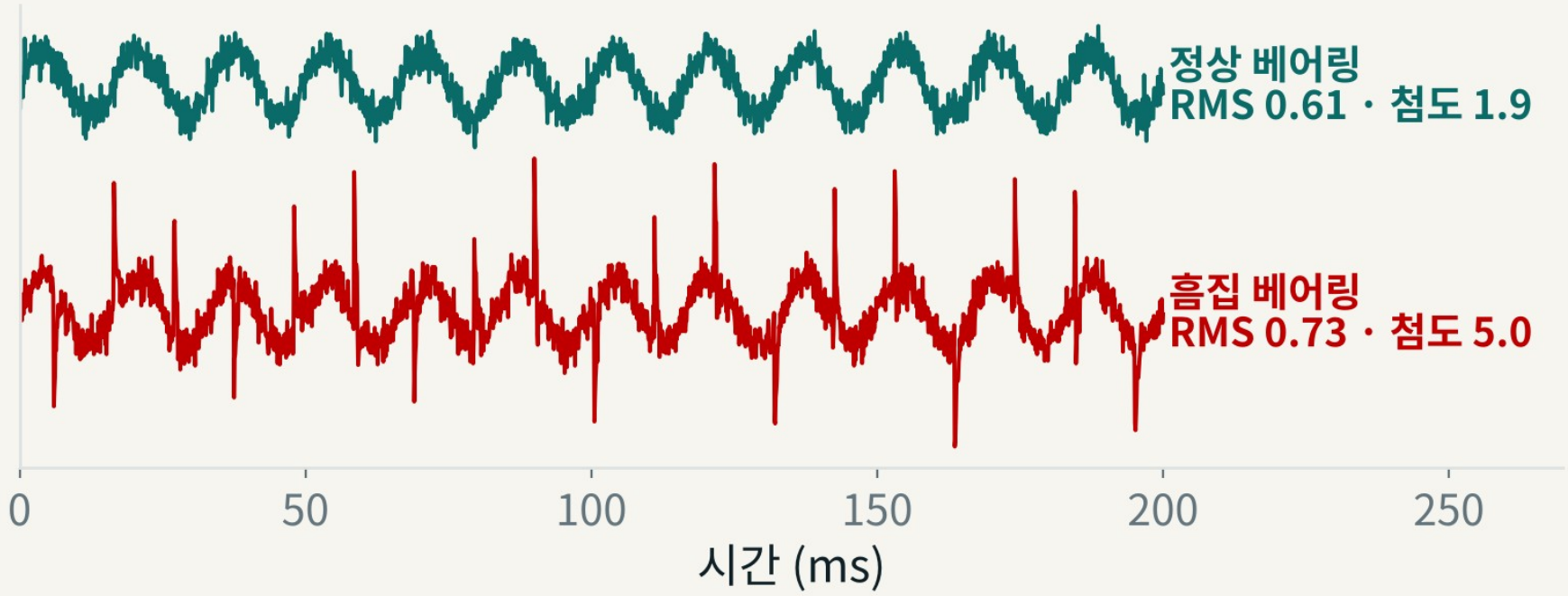
Claude Code · 예제 3 RMS·첨도

```
> 진동 지표를 계산해줘.
  1) [1, -1, 2, -2]의 RMS를 제곱→평균→제곱근 순서로 출력
  2) 60Hz 사인파+잡음(정상)과, 여기에 주기적 충격을
     더한 신호(흠집)를 만들어 RMS·첨도를 비교
  3) 두 파형을 vib.png로 저장, 난수 시드는 3으로 고정
  지표 함수는 vib.py에 rms(), kurtosis()로 분리해줘.
```

※ 난수 시드를 고정해야 다시 실행해도 같은 결과가 나와 서로 비교할 수 있다

흠집 신호에서는 첨도가 RMS보다 크게 뜁니다

vib.py 출력 — RMS([1, −1, 2, −2]) = 1.58 (손계산과 일치)



※ 예시 데이터 — RMS는 0.61 → 0.73(1.2배)인데 첨도는 1.9 → 5.0(2.6배)

확인 문제 4. 충격을 키우며 두 지표의 민감도를 잽니다

충격 크기만 0.5배·1배·2배로 바꿔 정상 대비 배수를 비교한다 (시드 3 고정)

RMS · 첨도 배수

- 충격 0.5배: 1.04 · 1.26
- 충격 1배: 1.19 · 2.63
- 충격 2배: 1.69 · 7.13

첨도가 더 민감 — 초기 결함은 첨도로

후속 프롬프트 · 문제 4

```
> vib.py에서 충격 크기만
    0.5배 · 1배 · 2배로 바꿔
  RMS와 첨도가 정상 대비
  각각 몇 배인지 표로 보여줘.
  결함 초기에 더 민감한 지표는?
```

예제 4. 손계산 값을 프롬프트에 함께 적어 대조시킵니다

η = 1,000시간 · β = 2 — 다르면 이유를 설명하라고 요청한다

Claude Code · 예제 4 Weibull

```
> Weibull 생존 확률을 계산해줘. η = 1,000시간, β = 2.
  1) R(500), R(1000)을 소수 둘째 자리로 출력
  2) β = 0.5, 1, 2의 생존 곡선을 한 그래프에 그려
     weibull.png로 저장 (두 점에 값 표시)
  3) R(t) = 0.5가 되는 시간(중앙 수명)도 계산
  손계산 값 0.78 · 0.37과 다르면 이유를 설명해줘.
```

※ 중앙 수명은 손으로 풀기 번거로운 역산 문제다 — 코드가 강점을 보이는 부분이다

중앙 수명은 약 833시간입니다

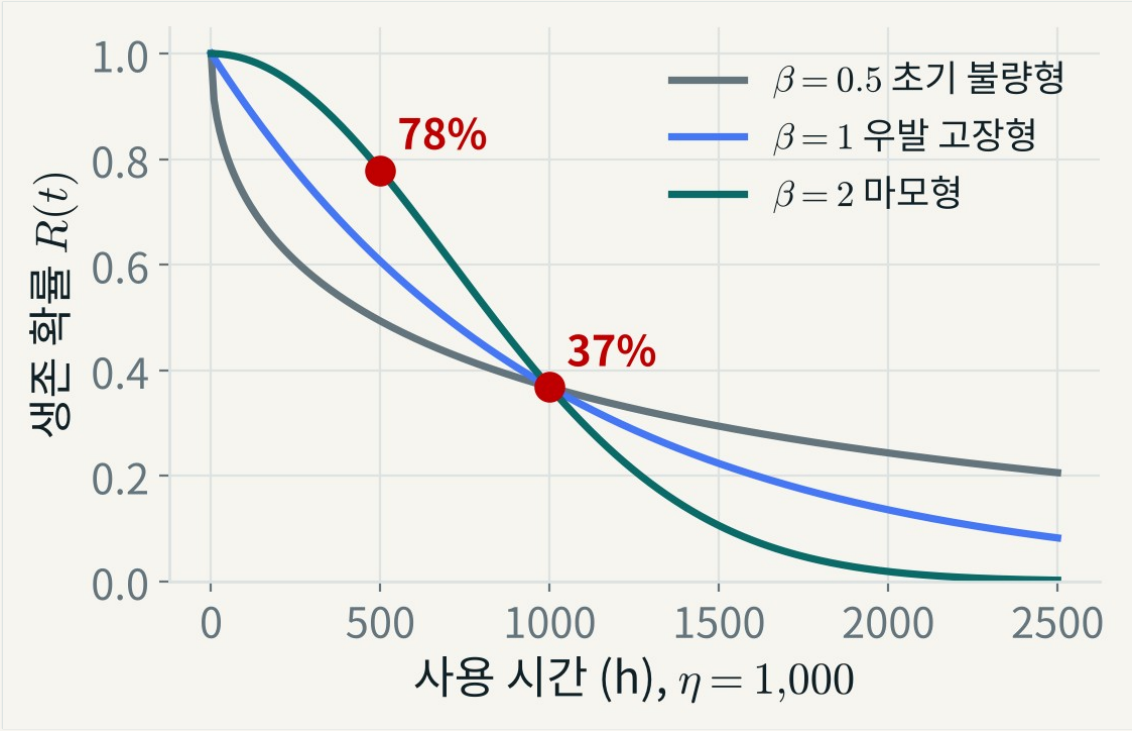
weibull.py 출력과 저장된 weibull.png

출력

R(500) = 0.78

- R(1000) = 0.37
- 중앙 수명 833h
- 손계산과 일치

η의 83%에서 절반



예제 5. 기준 계산과 시나리오를 한 번에 돌립니다

투자 3억 · 연 순효과 1.5억 · 3년 · 할인율 8% — 손익분기까지 요청한다

Claude Code · 예제 5 NPV·민감도

```
> 예지보전 투자의 NPV를 계산해줘.
  투자 3억 · 연 효과 1.8억(정지 1.2·정비 0.4·품질 0.2)
  운영비 0.3억 · 3년 · 할인율 8%
  1) 연도별 할인 계수·현재가치 표와 NPV 출력
  2) 정지 감소 절반·투자비 4억·운영비 2배·효과 +20%의
     NPV를 가로 막대그래프 npv.png로
  3) NPV가 0이 되는 투자비(손익분기)도 계산
```

※ 시나리오를 프롬프트에 나열하면 민감도 분석이 한 번의 실행으로 끝난다

손익분기 투자비는 약 3.87억 원입니다

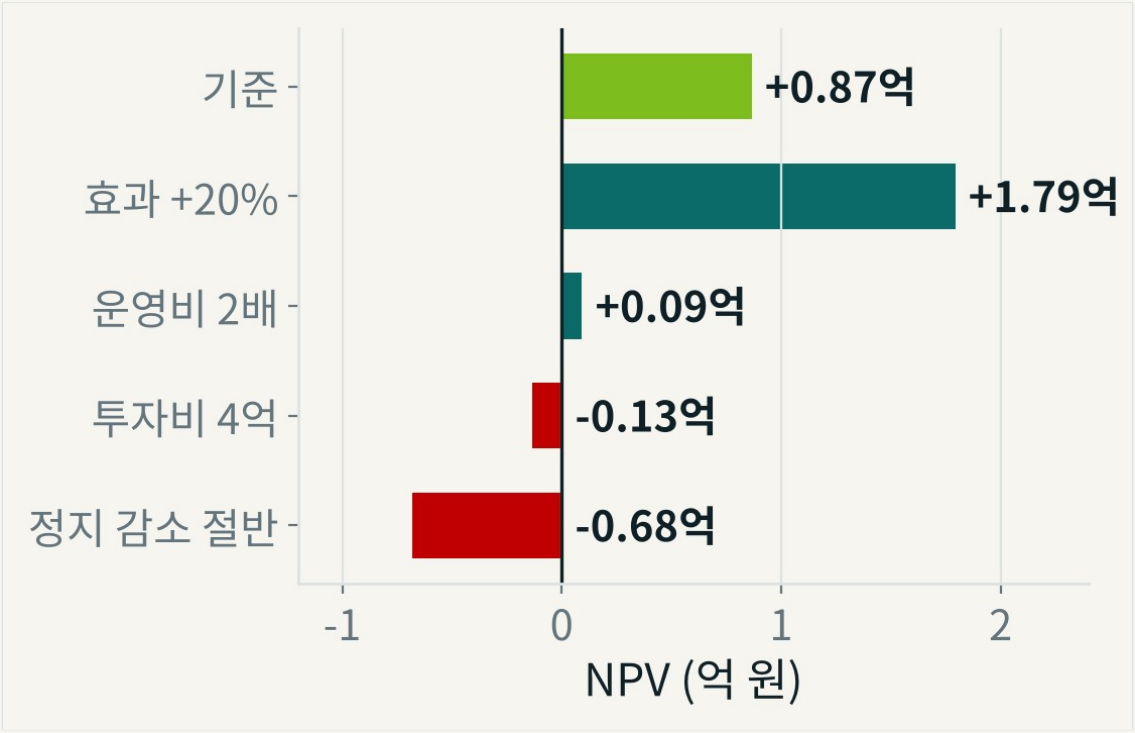
npv.py 출력과 저장된 npv.png

출력

NPV +0.87억

- 할인 계수 합 2.577
- 손익분기 3.87억
- 효과 절반 −0.68억

큰 가정부터 검증



확인 문제 5. 투자비를 바꾸면 필요한 순효과를 역산합니다

투자비 4억 원 시나리오와, NPV ≥ 0을 만드는 최소 연 순효과

실행 결과 · 해설

NPV ≈ −0.13억

- 최소 순효과 1.55억
- 지금 1.5억 → 부족
- 효과 3.5% 늘면 통과

적용 설비 확대 검토

후속 프롬프트 · 문제 5

```
> npv.py로 투자비 4억 시나리오만
  다시 계산해줘.
  NPV가 0 이상이 되려면
  연 순효과가 최소 얼마여야
  하는지도 알려줘.
```

결과를 쓰기 전에 세 가지를 확인합니다

Claude Code의 출력도 사람의 보고서처럼 검토 대상이다

대조

손계산과 assert

- 기대값을 프롬프트에
- 다르면 이유를 묻는다

재현

시드·입력 고정

- 난수 시드를 적는다
- 숫자는 단위까지

보관

코드를 .py로

- 다음 문제에 재사용
- 누구나 다시 실행

계산은 Claude Code가 빠르게, 판단은 숫자를 아는 사람이 합니다

손계산 → 프롬프트 → 검증의 순서를 지키면 AI 결과를 믿을 수 있습니다

손으로 이해하고,

코드로 반복하고,

숫자로 검증한다.

프롬프트는 계산을 맡기는 것이지
판단을 맡기는 것이 아니다.